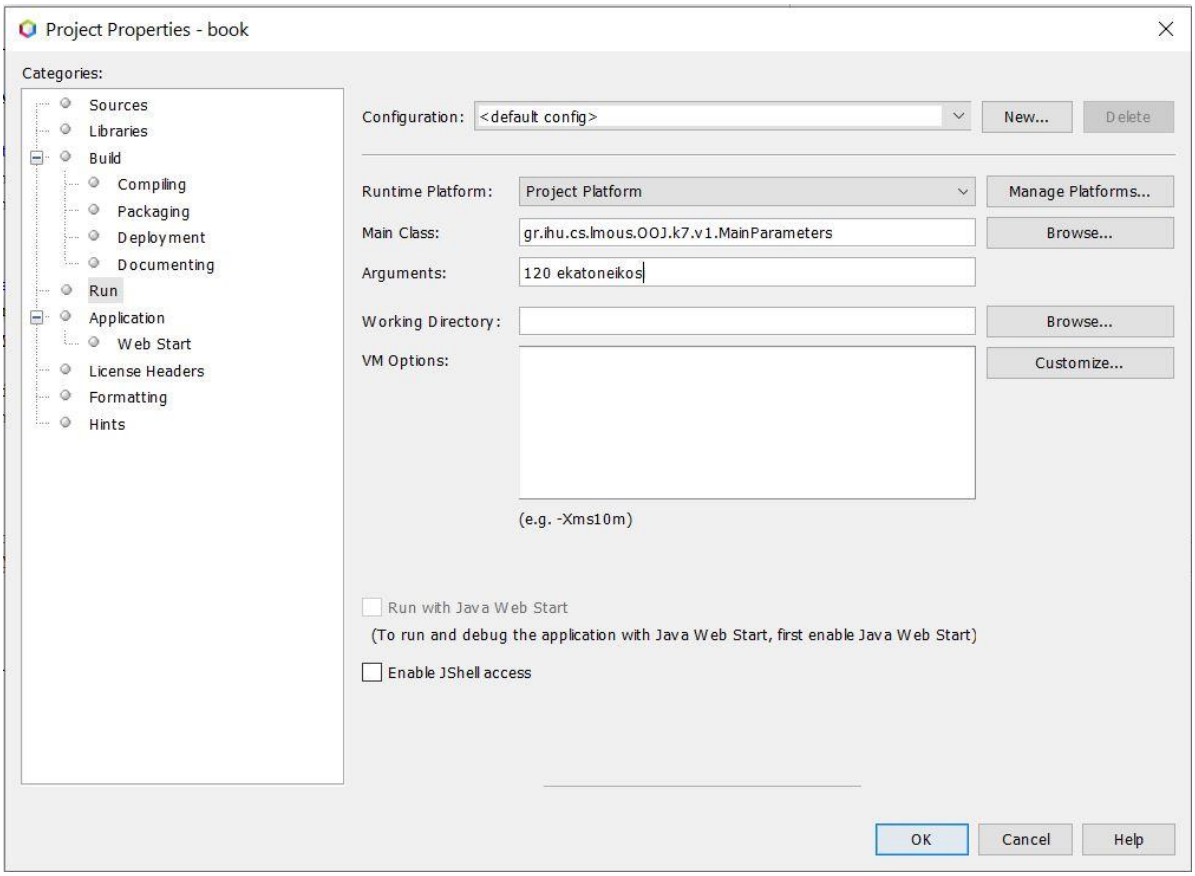


Εικόνα 7.1 *Καθορισμός παραμέτρων της main στο NetBeans*

Στην αριστερή πλευρά της φόρμας Project Properties είναι τοποθετημένη μία λίστα κάτω από την επικεφαλίδα Categories. Στη λίστα Categories, επιλέξτε Run όπως φαίνεται στην εικόνα 7.1. Στη δεξιά πλευρά της φόρμας βρείτε τα πεδία Main Class και Arguments και ενημερώστε τα κατάλληλα. Στο πεδίο Main Class εισάγετε την κλάση MainParameters με τον προσδιοριστή πακέτου όπως φαίνεται στην εικόνα 7.1. Ο προσδιοριστής πακέτου έχει το όνομα του πακέτου στο οποίο ανήκει η κλάση. Μπορείτε να χρησιμοποιήσετε το button Browse για διευκόλυνσή σας. Στο πεδίο Arguments, τοποθετήστε τις παραμέτρους που απαιτεί η εφαρμογή διαχωρισμένες με ένα διάστημα. Πατήστε το button OK ώστε να αποθηκευτούν οι νέες ιδιότητες.

Μεταβείτε στο project που έχετε ορίσει την κλάση, πατήστε δεξί κλικ και από το πτυσσόμενο μενού, επιλέξτε Run. Στο παράθυρο Output του NetBeans θα δείτε να εμφανίζεται το αποτέλεσμα κλήσης της MainParameters.

7.3.4.2 Κλήση από το powershell

Μεταβείτε στο Project Properties. Στο παράθυρο Categories, επιλέξτε Sources. Επιλέξτε τη διαδρομή στην οποία είναι αποθηκευμένο το project σας από το πεδίο Project Folder. Χρησιμοποιήστε Ctrl+c για να αντιγράψετε τη διαδρομή του project στο πρόχειρο των Windows (clipboard).

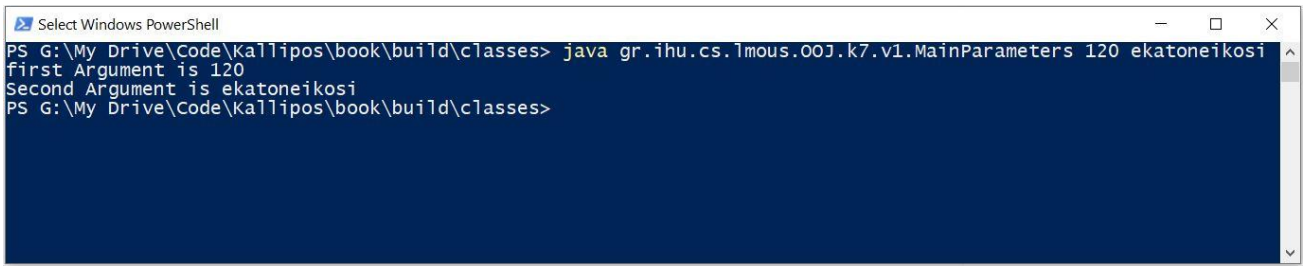
Ανοίξτε τον file explorer των Windows. Στη γραμμή εντολών δώστε Ctrl+V και Enter. Θα μεταβείτε έτσι στον φάκελο του project. Εκεί θα δείτε μια σειρά από φακέλους. Ο φάκελος build περιέχει φάκελο classes. Στον φάκελο classes είναι αποθηκευμένος ο bytecode της εφαρμογής σας οργανωμένος σε υποφακέλους ανάλογα με το πακέτο στο οποίο ανήκει η κάθε κλάση.

Μεταβείτε στον φάκελο classes. Ενώ βρίσκεστε στον φάκελο classes, πηγαίνετε στο μενού file του file explorer και κάντε κλικ στο Open Windows PowerShell. Θα ανοίξει το PowerShell με φάκελο εργασίας τον φάκελο classes.

Εκεί πληκτρολογήστε

```
Java packagePath.MainParameters first-argument second-argument
```

Στη θέση packagePath βάλτε το όνομα του πακέτου στο οποίο ανήκει η MainParameters. Η εικόνα 7.2 παρουσιάζει μια κλήση της MainParameter όταν αυτή έχει δηλωθεί να ανήκει στο πακέτο gr.ihu.cs.lmous.OOJ.k7.v1.



Εικόνα 7.2 *Κλήση εφαρμογής Java από το PowerShell των Windows.*

Όπως φαίνεται στην εικόνα 7.2, σε αυτό το παράδειγμα, η MainParameters καλείται με πρώτη παράμετρο το 112 και δεύτερη παράμετρο τη σειρά ekatondwdeka. Η λέξη Java στην αρχή της κλήσης ενεργοποιεί την JVM η οποία στην ουσία τρέχει την κλάση MainParameters.

7.4 Στατικές μεταβλητές

Η Java μας δίνει τη δυνατότητα να ορίσουμε στατικές μεταβλητές. Οι στατικές μεταβλητές ορίζονται στο επίπεδο της κλάσης και προσπελαύνονται μέσα από οποιαδήποτε συνάρτηση της κλάσης. Δεν υπάρχει δυνατότητα ορισμού στατικών μεταβλητών που είναι τοπικές μεταβλητές συναρτήσεων. Η πλήρης έννοια των στατικών μεταβλητών μπορεί να αναπτυχθεί μόνο στο πλαίσιο του Αντικειμενοστρεφούς μοντέλου όπου θα συζητηθούν και οι μη στατικές μεταβλητές και θα γίνει διαφοροποίηση μεταξύ στατικών και μη στατικών μεταβλητών.

Για την ώρα θα δούμε τη χρησιμότητα των στατικών μεταβλητών μέσα από ένα παράδειγμα αξιοποίησής των. Ας υποθέσουμε πως στην εφαρμογή που αναπτύσσουμε χρειαζόμαστε σε μορφή αλφαριθμητικής σειράς το άθροισμα ή και το γινόμενο πραγματικών αριθμών αποθηκευμένων σε κατάλληλο πίνακα. Σε αυτήν την περίπτωση είναι λογικό να υλοποιήσουμε δύο συναρτήσεις, μία που υπολογίζει και επιστρέφει το άθροισμα και μια για το γινόμενο. Θέλουμε όμως οι δύο συναρτήσεις να διαμορφώνουν τα αποτελέσματά τους με κοινό μοτίβο. Σκοπός μας είναι να εμφανίσουμε άθροισμα και γινόμενο και δεν επιθυμούμε να εμφανίζονται με διαφορετικό πλήθος δεκαδικών. Επομένως χρειαζόμαστε έναν τρόπο διαμόρφωσης του αποτελέσματος κοινό για τις δύο συναρτήσεις. Εδώ μας διευκολύνουν οι στατικές μεταβλητές της κλάσης:

```
import java.text.DecimalFormat;

public class StaticVars {

    public static String format; // "###.##"

    public static String sum(double[] in) {
        double sum = 0;
        for (double current : in) {
            sum += current;
        }
```

```
        DecimalFormat f = new DecimalFormat(format);
        return f.format(sum);
    }

    public static String product(double[] in) {
        double product = 1;
        for (double current : in) {
            product *= current;
        }
        DecimalFormat f = new DecimalFormat(format);
        return f.format(product);
    }

    public static void main(String[] args) {
        format = "###.##";
        double[] d = {5.6, 7.95, 3.23, 45, 96};
        System.out.println(sum(d));
        System.out.println(product(d));
    }
}
```

Κώδικας 7.11 *Χρήση στατικής μεταβλητής*

Στον κώδικα 7.11 δίνεται ο ορισμός της κλάσης StaticVars. Η κλάση περιλαμβάνει ορισμό των συναρτήσεων sum και product. Και οι δύο λαμβάνουν ως παράμετρο έναν πίνακα πραγματικών. Η sum υπολογίζει το άθροισμα των τιμών της παραμέτρου της και η product το γινόμενο. Και οι δύο υπολογίζουν το αποτέλεσμά τους και το επιστρέφουν ως String. Στην κλάση ορίζεται και η στατική μεταβλητή format. Σκοπός της είναι να ελέγχει τη διαμόρφωση κατά την εμφάνιση των πραγματικών μεταβλητών. Ως στατική μεταβλητή της κλάσης, η format μπορεί να προσπελαστεί από όλες τις συναρτήσεις της κλάσης. Στην main, πριν καλέσουμε τις μεθόδους, φροντίζουμε να ενημερώσουμε κατάλληλα την format. Στη συνέχεια, οι δύο μέθοδοι διαβάζουν την format και επιστρέφουν το αποτέλεσμά τους διαμορφωμένο σύμφωνα με το μοτίβο που αποθηκεύσαμε στην format.

Κατά μία έννοια, μια στατική μεταβλητή αποτελεί μία κοινή μνήμη για όλη την κλάση.

Ας υποθέσουμε τώρα πως χρειαζόμαστε την ίδια λειτουργικότητα, δηλαδή να εμφανίζουμε το άθροισμα και το γινόμενο των στοιχείων πινάκων πραγματικών και σε μία άλλη εφαρμογή. Έστω όμως πως στην άλλη εφαρμογή, χρειαζόμαστε διαφορετική διαμόρφωση, π.χ. θέλουμε τρία δεκαδικά ψηφία αντί για δύο. Με δεδομένο ότι οι συναρτήσεις sum και product αλλά και η σειρά format είναι δημόσια μέλη της κλάσης, η λύση είναι απλή και παρατίθεται στον κώδικα 7.12.

```
public class AnotherApp {

    public static void main(String[] args) {
        StaticVars.format = "###.###";
        double[] d = {5.675, 7.915, 3.213, 45, 0.96};
        System.out.println(StaticVars.sum(d));
        System.out.println(StaticVars.product(d));
    }
}
```

Κώδικας 7.12 *Χρήση στατικής μεταβλητής από άλλη κλάση*

Στον κώδικα 7.12, η main της κλάσης AnotherApp δίνει τιμή στη στατική μεταβλητή format της StaticVars. Στη συνέχεια καλεί τις sum και product της StaticVars προκειμένου να εμφανίσει το άθροισμα και το γινόμενο των στοιχείων του πίνακα d.

Όπως θα αντιληφθήκατε ήδη, προκειμένου να προσπελάσουμε μια στατική συνάρτηση ή μια στατική μεταβλητή από κλάση διαφορετική από αυτήν που ορίζονται πρέπει να προηγηθεί του ονόματός τους, το

όνομα της κλάσης στην οποία ορίζονται. Σε αυτήν τη χρήση, το όνομα της κλάσης θεωρείται ένα προσδιοριστικό ονόματος (name specifier).

Αν έχουμε σύγκρουση μεταξύ αναγνωριστικού στατικής μεταβλητής μιας κλάσης με τοπική μεταβλητή μέσα σε συνάρτηση της ίδιας κλάσης, τότε θα πρέπει να χρησιμοποιήσουμε τον προσδιοριστή ονόματος κατά την αναφορά στη στατική μεταβλητή.

7.5 Απροσδόκητα λάθη

Ας υποθέσουμε πως θέλουμε να αναπτύξουμε μια συνάρτηση που υπολογίζει το παραγοντικό μιας ακέραιας τιμής. Το πρώτο βήμα βέβαια θα είναι να βεβαιωθούμε πως γνωρίζουμε τον σωστό ορισμό του παραγοντικού.

Για τη διευκόλυνση της συζήτησής μας παραθέτουμε τον ορισμό.

Το παραγοντικό ενός μη αρνητικού ακέραιου n, συμβολίζεται με n! και είναι ίσο με 1 αν n=0 ή n=1, ενώ αν n>1, είναι το γινόμενο όλων των θετικών ακέραιων που είναι μικρότεροι ή ίσοι με το n.

Με δεδομένο τον ορισμό, η ανάπτυξη της συνάρτησης είναι σχετικά απλή. Ο κώδικας 7.13 μας δίνει τη συνάρτηση του παραγοντικού.

```
public class UnexpectedErrors {

    static long factorial(long n) { //Κώδικας 7.13
        if (n == 0 || n == 1) {
            return 1;
        }
        long rVal = 1;
        for (long i = 2; i <= n; i++) {
            rVal *= i;
        }
        return rVal;
    }
}
```

Κώδικας 7.13 *Υπολογισμός παραγοντικού*

Η συνάρτηση factorial είναι τύπου long καθώς το παραγοντικό μεγαλώνει γρήγορα σε σχέση με το όρισμά του. Για παράδειγμα, 3! = 6, 6! = 720 και 9! = 362880. Στην αρχή της συνάρτηση factorial, ελέγχεται το n καθώς αν είναι 0 ή 1 δεν απαιτούνται περαιτέρω υπολογισμοί μια και για αυτές τις τιμές του n, το n! είναι ίσο με 1. Στη συνέχεια αρχικοποιείται η rVal στην τιμή 1 και πολλαπλασιάζεται διαδοχικά με όλους τους ακέραιους που είναι μεγαλύτεροι ή ίσοι του 2 και μικρότεροι ή ίσοι του n. Με την έξοδο από τον βρόχο for, η rVal αναπαριστά την απαιτούμενη τιμή την οποία η συνάρτηση επιστρέφει.

Αν τρέξετε την factorial, θα διαπιστώσετε ότι επιστρέφει τη σωστή τιμή για όλες τις μη αρνητικές τιμές της παραμέτρου n.

Τι θα συμβεί όμως αν η factorial κληθεί με πραγματική παράμετρο έναν αρνητικό ακέραιο; Καταρχάς, δεν θα επιστρέψει από την αρχική if. Στη συνέχεια θα αρχικοποιήσει την rVal στο 1. Στη συνέχεια δεν θα μπει στον βρόχο for, καθώς το θετικό i δεν είναι μικρότερο ή ίσο από οποιονδήποτε αρνητικό αριθμό και τέλος θα επιστρέψει το rVal του οποίου η τιμή παραμένει 1. Με λίγα λόγια, η factorial του κώδικα 7.13 επιστρέφει την τιμή 1 ως αποτέλεσμα υπολογισμού του παραγοντικού οποιουδήποτε αρνητικού ακέραιου.

Αυτό όμως είναι λάθος, μιας και δεν ορίζεται παραγοντικό για αρνητικούς ακέραιους. Πρόκειται μάλιστα για σοβαρό λάθος με απρόβλεπτες συνέπειες κατά την εκτέλεση της εφαρμογής που το περιέχει.

Μπορεί να αναρωτιέστε γιατί να κληθεί η factorial με αρνητικό ακέραιο. Ενδεχομένως, η factorial να καλείται με όρισμα κάποια μεταβλητή η οποία να προκύπτει από κάποιους υπολογισμούς. Ενδεχομένως, το λάθος να ξεκινά από εκεί. Γενικά, στις ρεαλιστικές εφαρμογές όπου εμπλέκονται πάρα πολλές μεταβλητές και πάρα πολλοί και σύνθετοι υπολογισμοί, τέτοιου είδους λάθη είναι σχεδόν αδύνατο να

αποκλειστούν. Εκείνο όμως που μπορεί να γίνει είναι να εντοπιστούν εγκαίρως και να διορθωθούν. Σε αυτό μας βοηθάνε οι εξαιρέσεις (Exceptions) της Java.

Η διαχείριση των απροσδόκητων λαθών είναι μεγάλο κομμάτι κάθε γλώσσας προγραμματισμού και της Java. Αντίστοιχα, πολλές είναι και οι κλάσεις που αφορούν τη διαχείριση των λαθών. Υπάρχουν διαχειρίσιμα και μη διαχειρίσιμα λάθη. Λεπτομέρειες για τη διαχείριση των λαθών συζητάμε στην ενότητα 16. Για την ώρα μας αρκεί μια προσέγγιση που αν και απλή μας βοηθάει να προστατευτούμε από τα απροσδόκητα λάθη στον κώδικά μας. Η προσέγγιση λοιπόν που προτείνουμε εδώ βασίζεται στην παραγωγή μιας εξαίρεσης τύπου RuntimeException. Με άλλα λόγια, στην αρχή της συνάρτησης factorial θα ελέγξουμε το όρισμα και αν βρεθεί αρνητικό θα παράγουμε μια εξαίρεση τύπου RuntimeException. Η παραγωγή της εξαίρεσης θα προκαλέσει τερματισμό του προγράμματος με εμφάνιση κατάλληλου μηνύματος. Έτσι θα βοηθηθούμε να εντοπίσουμε το πρόβλημα και να το διορθώσουμε.

Καλό είναι αυτός που καλεί τη συνάρτησή μας να γνωρίζει πως η συνάρτηση ενδέχεται να παράγει εξαίρεση. Έτσι, η Java υποστηρίζει τη δήλωση των εξαιρέσεων που ενδέχεται να παράγει μια συνάρτηση στην επικεφαλίδα της. Ο κώδικας 7.14 παρουσιάζει την factorial με ενσωματωμένες τις παρατηρήσεις που κάναμε σε αυτήν την ενότητα.

```
public class UnexpectedErrors {

    static long factorial(long n) throws RuntimeException { //Κώδικας 7.14
        if (n < 0) {
            throw new RuntimeException();
        }
        if (n == 0 || n == 1) {
            return 1;
        }
        long rVal = 1;
        for (long i = 2; i <= n; i++) {
            rVal *= i;
        }
        return rVal;
    }
}
```

Κώδικας 7.14 *Η factorial με έλεγχο της παραμέτρου*

Όπως φαίνεται στον κώδικα 7.14, η παραγωγή της εξαίρεσης γίνεται με τη λέξη-κλειδί throw. Με τη βοήθεια της λέξης-κλειδί throws δηλώνεται προαιρετικά στην επικεφαλίδα της συνάρτησης το γεγονός ότι η συνάρτηση ενδέχεται να παράγει εξαίρεση τύπου RuntimeException.

Τρέξτε τη νέα έκδοση της factorial με αρνητικό όρισμα και μελετήστε το αποτέλεσμα.

7.6 Χρήσιμες συναρτήσεις

Σε αυτήν την ενότητα παρουσιάζουμε συναρτήσεις ενσωματωμένες στο βασικό πλαίσιο της Java που συχνά μας είναι χρήσιμες. Όπως θα έχετε ήδη αντιληφθεί, κάθε συνάρτηση στην Java ανήκει και σε μία κλάση. Επιπρόσθετα λοιπόν κάποιων συναρτήσεων της βιβλιοθήκης της Java που έχουμε ήδη δει, στην ενότητα αυτή θα δούμε κάποιες συναρτήσεις που ανήκουν στις κλάσεις Math και String.

7.6.1 Η κλάση Math

Η κλάση Math περιλαμβάνει μια σειρά από στατικές συναρτήσεις και στατικές σταθερές μεταβλητές που μας επιτρέπουν να κάνουμε βασικούς μαθηματικούς υπολογισμούς. Παρουσιάζουμε εδώ μέρος των συναρτήσεων και σταθερών μεταβλητών της Math. Αυτές που θεωρούμε πως είναι πιο αναγκαίες στα πλαίσια αυτού του εγχειριδίου. Για κάθε μια συνάρτηση παρουσιάζουμε πρώτα τη διεπαφή της ακολουθούμενη από μια μικρή περιγραφή και στη συνέχεια επιδεικνύουμε τον τρόπο κλήσης της.
Η σταθερά E αναπαριστά το e, τη βάση των φυσικών λογάριθμων. Η διεπαφή της έχει ως εξής:

```
public static final double E
```

Η σταθερά PI αναπαριστά το π, την αναλογία της περιφέρειας ενός κύκλου προς τη διάμετρο του κύκλου. Η διεπαφή της είναι:

```
public static final double PI
```

Η συνάρτηση toRadians μετατρέπει τις μοίρες σε ακτίνια.

```
public static double toRadians(double angdeg)
```

Η παράμετρος angdeg αναπαριστά τις μοίρες μιας γωνίας. Η συνάρτηση επιστρέφει τα ακτίνια της ίδιας γωνίας.

Η συνάρτηση toDegrees μετατρέπει τα ακτίνια σε μοίρες.

```
public static double toDegrees(double angrad)
```

Η παράμετρος angrad αναπαριστά τα ακτίνια μιας γωνίας. Η συνάρτηση επιστρέφει τις μοίρες της ίδιας γωνίας.

Η συνάρτηση sin επιστρέφει το ημίτονο μιας γωνίας.

```
public static double sin(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε το ημίτονο μιας γωνίας 30 μοιρών, δεν έχετε παρά να καλέσετε

```
Math.sin(Math.toRadians(30))
```

Η συνάρτηση cos επιστρέφει το συνημίτονο μιας γωνίας.

```
public static double cos(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε το συνημίτονο των 30 μοιρών καλέστε

```
Math.cos(Math.toRadians(30))
```

Η συνάρτηση tan επιστρέφει την εφαπτομένη γωνίας.

```
public static double tan(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε την εφαπτομένη των 30 μοιρών καλέστε

```
Math.tan(Math.toRadians(30))
```

Η συνάρτηση log επιστρέφει τον φυσικό λογάριθμο της παραμέτρου της.

```
public static double log(double a)
```

Για να υπολογίσετε τον φυσικό λογάριθμο του 90, καλέστε

```
Math.log(90)
```

Η συνάρτηση log10 επιστρέφει τον λογάριθμο με βάση το 10.

```
public static double log10(double a)
```

Για να υπολογίσετε τον λογάριθμο με βάση 10 του 90, καλέστε

```
Math.log10(90)
```

Η συνάρτηση pow επιστρέφει τη δύναμη ενός αριθμού υψωμένου σε έναν εκθέτη.

```
public static double pow(double a,  double b)
```

Η παράμετρος a αναπαριστά τη βάση και η b τον εκθέτη. Για να υπολογίσετε την παράσταση 5.24 καλέστε

Math.pow(5.2, 4)

Η συνάρτηση sqrt επιστρέφει την τετραγωνική ρίζα του ορίσματός της.

```
public static double sqrt(double a)
```

Για να υπολογίσετε την τετραγωνική ρίζα του 81, καλέστε

```
Math.sqrt(81)
```

Η συνάρτηση ceil επιστρέφει τη μικρότερη ακέραια τιμή που είναι μεγαλύτερη ή ίση με την παράμετρό της.

```
public static double ceil(double a)
```

Η κλήση ceil(9.1) επιστρέφει 10d, ενώ η κλήση ceil(9.0) επιστρέφει 9d.

Η συνάρτηση floor επιστρέφει τη μεγαλύτερη ακέραια τιμή που είναι μικρότερη ή ίση με την παράμετρό της.

```
public static double floor(double a)
```

Η κλήση floor(9.1) επιστρέφει 9d όπως και η κλήση floor(9.0).

Η συνάρτηση rint επιστρέφει την ακέραια τιμή που είναι πλησιέστερη στην τιμή της παραμέτρου της.

```
public static double rint(double a)
```

Η κλήση rint(9.4) επιστρέφει 9d και η κλήση rint(9.6) επιστρέφει 10d. Αν η τιμή της παραμέτρου βρίσκεται ακριβώς στη μέση μεταξύ δύο ακέραιων τιμών, τότε η rint επιστρέφει τον ακέραιο που είναι άρτιος. Επομένως, η κλήση rint(9.5) θα επιστρέψει 10d όπως ακριβώς και η κλήση rint(10.5).

Η συνάρτηση round επιστρέφει την ακέραια τιμή που είναι πλησιέστερη στην τιμή της παραμέτρου της. Η συνάρτηση είναι υπερφορτωμένη. Αν κληθεί με όρισμα double, επιστρέφει long και αν κληθεί με όρισμα float, επιστρέφει int. Αν η τιμή της παραμέτρου βρίσκεται ακριβώς στη μέση μεταξύ δύο ακέραιων τιμών, τότε η round επιστρέφει τον μεγαλύτερο από τους πλησιέστερους ακέραιους.

```
public static long round(double a)
public static int round(float a)
```

Η κλήση Math.round(9.4) επιστρέφει 9 και η κλήση Math.round(9.5) επιστρέφει 10.

Η συνάρτηση max επιστρέφει το μεγαλύτερο από τα ορίσματά της. Η συνάρτηση είναι υπερφορτωμένη και υποστηρίζει όλους τους αριθμητικούς τύπους. Οι διαθέσιμες διεπαφές έχουν ως εξής:

```
public static double max(double a, double b)
public static float max(float a, float b)
public static int max(int a, int b)
public static long max(long a, long b)
```

Αν καλέσετε την max με ορίσματα διαφορετικού τύπου, τότε θα κληθεί η έκδοση που είναι συμβατή και με τους δύο τύπους. Για παράδειγμα, αν την καλέσετε με double και long, τότε θα κληθεί η max(double, double). Έτσι η έκφραση

```
double m=Math.max(1d, 2L);
```

είναι σωστή. Ενώ η έκφραση

```
long m=Math.max(1d, 2L);
```

είναι λάθος.

Ακριβώς ανάλογη με την max είναι η min.

Η συνάρτηση abs επιστρέφει την απόλυτη τιμή της παραμέτρου της. Η abs είναι επίσης υπερφορτωμένη για όλους τους αριθμητικούς τύπους. Οι διαθέσιμες διεπαφές έχουν ως εξής:

```
public static double abs(double a)
public static float abs(float a)
public static long abs(long a)
public static int abs(int a)
```

7.6.2 Οι κλάσεις των θεμελιωδών τύπων

Έχουμε ήδη εξηγήσει πως για κάθε θεμελιώδη τύπο, η Java παρέχει μια αντίστοιχη κλάση. Κάθε μια τέτοια κλάση παρέχει σταθερές και συναρτήσεις που είναι χρήσιμες κατά την εκτέλεση εργασιών με θεμελιώδεις τύπους. Σε αυτήν την ενότητα συγκεντρώνουμε τις σημαντικότερες στατικές συναρτήσεις αυτών των κλάσεων.

7.6.2.1 Η κλάση Character

Η συνάρτηση isDigit επιστρέφει true αν η παράμετρος της αναπαριστά ψηφίο σύμφωνα με την κωδικοποίηση Unicode. Επομένως, εκτός από τα ψηφία που χρησιμοποιούνται στη Δυτική γραφή, υποστηρίζονται και τα ψηφία από διάφορους πολιτισμούς. Για παράδειγμα, οι χαρακτήρες '٢', '٣' και '٤' αποτελούν ψηφία στο αραβικό σύστημα αρίθμησης. Η διεπαφή της συνάρτησης είναι

```
public static boolean isDigit(char ch)
```